

1. Objetivos da atividade

Ao final da atividade, o estudante deverá ser capaz de:

- Compreender o conceito de Máquina de Turing.
- Identificar os principais componentes de uma Máquina de Turing.
- Compreender a importância das Máquinas de Turing para a computação.
- Construir e simular uma Máquina de Turing simples.
- Reconhecer que existem problemas que não podem ser resolvidos por algoritmos, compreendendo os limites da computação.

2. Conteúdo

- Conceito de Máquina de Turing.
- Fita, cabeça de leitura/escrita e estados.
- Alfabeto e regras de transição.
- Funcionamento de uma Máquina de Turing.
- Simulação computacional.
- Computabilidade e limites computacionais.

3. Desenvolvimento da atividade

Etapa 1 — Introdução

Após assistir ao vídeo e estudar o material disponibilizado, responda às questões abaixo.

1. O que é uma Máquina de Turing?

São construções matemáticas abstratas que nos ajudam a descrever rigorosamente o que significa computação.

2. Quais são os principais componentes de uma Máquina de Turing?

$(Q, \Sigma, \Gamma, \delta, q_0, q_{a\Box}, q_{a\Box})$

3. Qual é a importância das Máquinas de Turing para a computação?

O que chamamos de computador moderno tem uma característica muito importante: ela consegue carregar e armazenar configurações, ou o que chamamos de programas. A definição é se ela é ou não é uma máquina universal de Turing, particularmente com a característica de não distinção entre programa e dados no armazenamento, na tal fita infinita, o que permite a universalidade dele poder simular outra máquina de Turing.

4. Qual é a relação entre Máquina de Turing e algoritmo?

Turing foi ter definido com rigor matemático o que é um programa, na forma da máquina de Turing, configurado como transições entre estados finitos.

Etapa 2 — Simulação

Utilize um software ou simulador de Máquina de Turing indicado pelo professor.

Crie uma máquina capaz de reconhecer palavras da forma:

$0^n 1^n$

Exemplos aceitos

- 01
- 0011
- 000111
- 00001111

Exemplos rejeitados

- 0
- 1
- 001
- 011
- 00111

Desafio: A máquina deverá verificar se existe a mesma quantidade de símbolos **0** e **1**, seguindo a lógica de funcionamento de uma Máquina de Turing.

Etapa 3 — Registro da simulação

Após executar a máquina, registre:

- A palavra utilizada como entrada.
- Os estados percorridos.
- O resultado da execução: **ACEITA** ou **REJEITA**.
- Uma captura de tela da simulação.

Realize pelo menos **3 testes**:

- 2 entradas que devem ser aceitas.
- 1 entrada que deve ser rejeitada.

Registro dos testes

Teste	Entrada	Resultado esperado	Resultado obtido	Estados percorridos
1	0011	ACEITA	ACEITA	start → find_1 → find_1 → rewind → rewind → start → find_1 → find_1 → rewind → rewind → start → check_all → check_all → accept
2	000111	ACEITA	ACEITA	start → find_1 → find_1 → find_1 → rewind → rewind → rewind → start → find_1 → find_1 → find_1 → rewind → rewind → rewind → start → find_1 → find_1 → find_1 → rewind → rewind → rewind → start → check_all → check_all → check_all → accept
3	0101	REJEITA	REJEITA	start → find_1 → rewind → start → check_all (Trava / Rejeita)

Descrição da Máquina de Turing criada:

A máquina opera pelo método de pareamento em ziguezague, casando cada 0 da esquerda com um 1 correspondente da direita por meio de marcadores:

Marca o 0: Lê o primeiro 0 não processado à esquerda e o substitui por X.

Busca o 1 correspondente: Move a cabeça de leitura para a direita até encontrar o primeiro 1 disponível e o substitui por Y.

Rebobina a fita: Retorna para a esquerda até encontrar o último X gravado e avança uma posição para reiniciar o ciclo no próximo 0.

Valida o encerramento: Quando acabam os zeros (encontra Y logo de início), varre a fita para a direita conferindo se só restam Ys até o espaço em branco.

Se todos os pares casarem perfeitamente na ordem correta, a máquina para no estado de aceitação. Se houver excesso de zeros, excesso de uns ou inversão na ordem dos caracteres, ela trava por falta de transição, rejeitando a palavra.

Etapa 4 — Reflexão sobre os limites computacionais

Uma Máquina de Turing consegue resolver qualquer problema? Explique com suas palavras por que existem problemas que não podem ser resolvidos por algoritmos.

Na teoria sim, na prática atingível humana não. Bem, não temos hardwares suficientes para resolver um problema "Turing complete" em qualquer linguagem existente.

6. Questão final

Problema para reflexão:

Imagine que você recebeu um problema computacional muito complexo. Como saber se ele é apenas difícil de resolver ou se, na verdade, não existe nenhum algoritmo capaz de resolvê-lo para todos os casos?

Explique utilizando os conceitos estudados sobre **Máquinas de Turing, computabilidade e limites computacionais**.

Para saber se um problema é apenas muito difícil ou se ele é impossível de resolver, a primeira coisa é separar complexidade de computabilidade.

Um problema difícil (intratável) tem solução algorítmica conhecida, mas exige poder de processamento ou tempo inviável à medida que a entrada cresce (como problemas fatoração ou o Caixeiro-Viajante). O algoritmo existe, o desafio é apenas o custo computacional. Já um problema incomportável (indecidível) é aquele em que não existe — e a lógica matemática prova que nunca existirá — um algoritmo capaz de responder

corretamente e parar para todos os casos possíveis. Isso não é uma limitação de hardware ou de otimização de código, mas sim um limite da própria lógica. Como o Akita pontua no vídeo ao comparar calculadoras com computadores, uma calculadora comum possui funções fixas gravadas diretamente nos circuitos. Já a Máquina de Turing Universal — que inspirou a arquitetura de Von Neumann — trouxe a ideia de armazenar os dados e o próprio programa no mesmo lugar (na fita/memória). Essa capacidade de ler e executar outros programas confere universalidade ao computador, mas introduz um paradoxo central: o Problema da Parada. Alan Turing provou por contradição que é impossível construir um algoritmo geral capaz de analisar qualquer programa arbitrário com uma entrada e determinar, com certeza e em tempo finito, se ele vai parar ou rodar para sempre. Sendo assim, para descobrir se o problema recebido não tem solução para todos os casos, a técnica padrão é a redução ao Problema da Parada: se for demonstrado que resolver esse problema específico implicaria, como consequência lógica, conseguir resolver o Problema da Parada (ou prever qualquer comportamento em tempo de execução de um programa arbitrário, como dita o Teorema de Rice), então fica provado que ele é incomputável.